Experiment no: - 3

Experiment Name: - Basic SQL Retrieval Statements-Selection (where), Projection (Select) and functions.

Aim: -Performing practical by using Basic Retrieval Statements**,** restricting data and built-in Function

PART B
(PART B: TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in charge faculties at the end of the practical in case the there is no Black board access available)

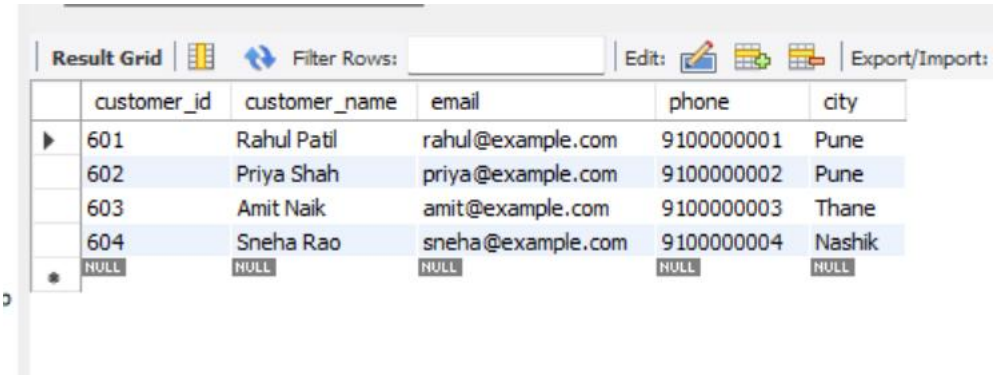
Roll. No.: C-60	NAME: Avdhut Madhavi
Class: C	Batch:C2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a SQL statement and check the output. (Put the query and its output after each question.)

To perform practical operations using basic SQL retrieval statements, restrict data using the WHERE clause, and apply built-in functions on the Population Management System database**.**

1. Display all records from the customer table

```
SELECT * FROM customer;
```

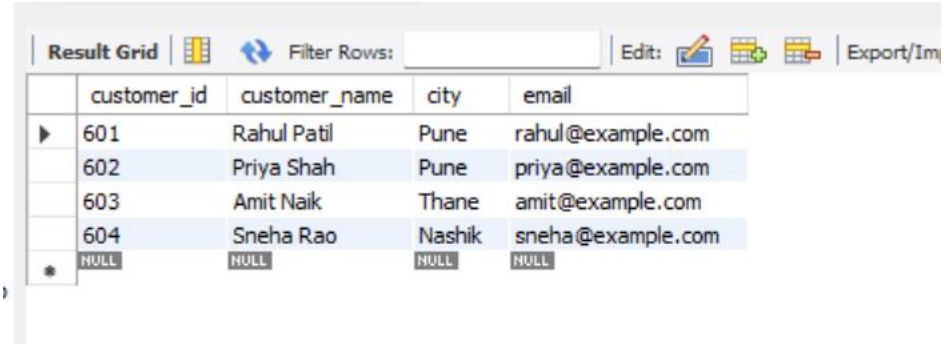


customer_id	customer_name	email	phone	city
601	Rahul Patil	rahul@example.com	9100000001	Pune
602	Priya Shah	priya@example.com	9100000002	Pune
603	Amit Naik	amit@example.com	9100000003	Thane
604	Sneha Rao	sneha@example.com	9100000004	Nashik
NULL	NULL	NULL	NULL	NULL

Explanation: Displays all columns and records in the customer table.

2. Display selected columns (projection)

```
SELECT customer_id, customer_name, city, email
FROM customer;
```

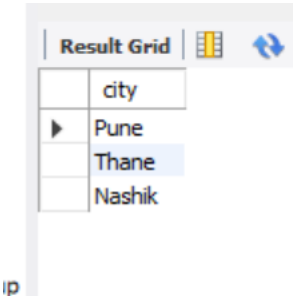


customer_id	customer_name	city	email
601	Rahul Patil	Pune	rahul@example.com
602	Priya Shah	Pune	priya@example.com
603	Amit Naik	Thane	amit@example.com
604	Sneha Rao	Nashik	sneha@example.com
NULL	NULL	NULL	NULL

Explanation: Displays only the customer ID, name, city and email columns.

3. Display unique customer genders

```
SELECT DISTINCT city
FROM customer;
```

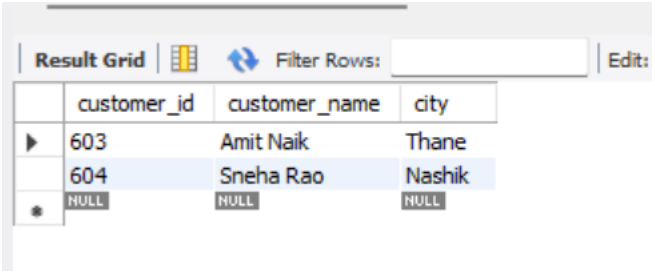


city
Pune
Thane
Nashik

Explanation: Displays each unique city without duplicates.

4.Display customers whose ID is greater than 602

```
SELECT customer_id, customer_name, city
FROM customer
WHERE customer_id > 602;
```

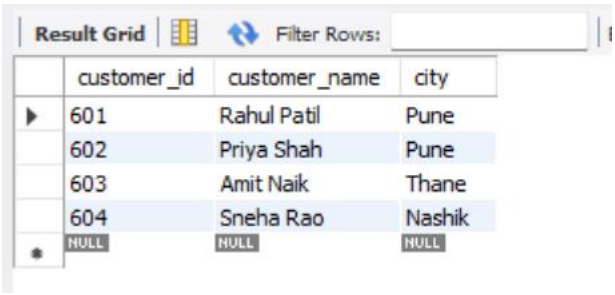


customer_id	customer_name	city
603	Amit Naik	Thane
604	Sneha Rao	Nashik
NULL	NULL	NULL

Explanation: Displays customers whose IDs are greater than 602.

5. Display customers whose IDs are between 601 and 604

```
SELECT customer_id, customer_name, city
FROM customer
WHERE customer_id BETWEEN 601 AND 604;
```



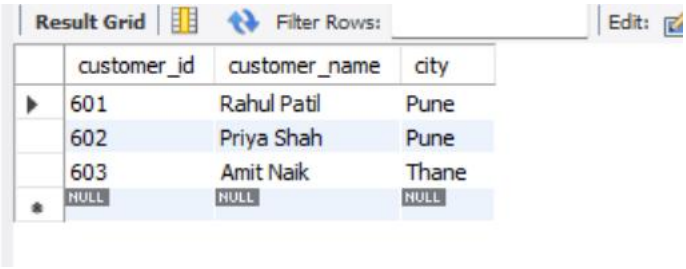
Result Grid | Filter Rows:

customer_id	customer_name	city
601	Rahul Patil	Pune
602	Priya Shah	Pune
603	Amit Naik	Thane
604	Sneha Rao	Nashik
NULL	NULL	NULL

Explanation: BETWEEN includes both boundary values.

6. Display customers from selected cities

```
SELECT customer_id, customer_name, city
FROM customer
WHERE city IN ('Mumbai', 'Pune', 'Thane');
```



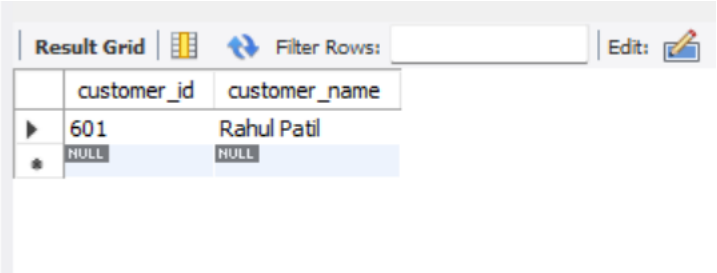
Result Grid | Filter Rows: | Edit:

customer_id	customer_name	city
601	Rahul Patil	Pune
602	Priya Shah	Pune
603	Amit Naik	Thane
NULL	NULL	NULL

Explanation: Retrieves customers whose addresses match any of the listed cities.

7. Search customer names using LIKE

```
SELECT customer_id, customer_name
FROM customer
WHERE customer_name LIKE 'R%';
```



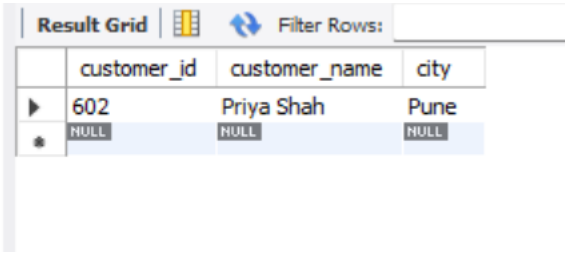
Result Grid | Filter Rows: | Edit:

customer_id	customer_name
601	Rahul Patil
NULL	NULL

Explanation: Displays customers whose names begin with the letter R.

8. Use logical operators AND and OR

```
SELECT customer_id, customer_name, city
FROM customer
WHERE (city = 'Mumbai' OR city = 'Pune')
AND customer_id > 601;
```

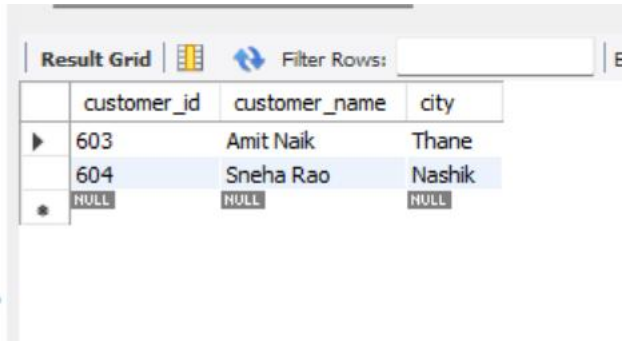


customer_id	customer_name	city
602	Priya Shah	Pune
NULL	NULL	NULL

Explanation: Displays customers from Mumbai or Pune whose IDs are greater than 601.

9. Use the NOT operator

```
SELECT customer_id, customer_name, city
FROM customer
WHERE city NOT IN ('Mumbai', 'Pune');
```

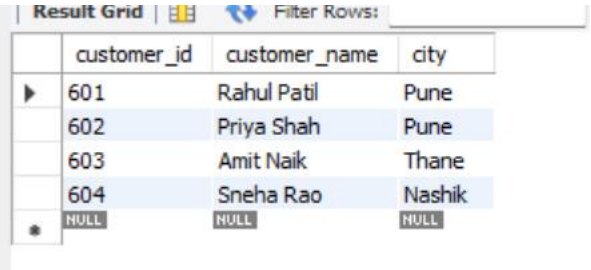


customer_id	customer_name	city
603	Amit Naik	Thane
604	Sneha Rao	Nashik
NULL	NULL	NULL

Explanation: Displays customers whose addresses are neither Mumbai nor Pune**.**

10. Sort customers by customer ID

```
SELECT customer_id, customer_name, city
FROM customer
ORDER BY customer_id ASC;
```

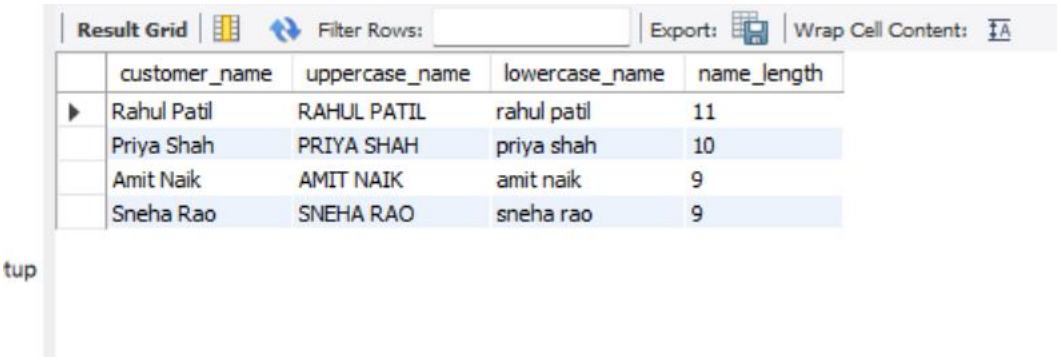


customer_id	customer_name	city
601	Rahul Patil	Pune
602	Priya Shah	Pune
603	Amit Naik	Thane
604	Sneha Rao	Nashik
NULL	NULL	NULL

Explanation: Displays customers in ascending order of customer ID.

11. Use character functions

```
SELECT
   customer_name,
   UPPER(customer_name) AS uppercase_name,
   LOWER(customer_name) AS lowercase_name,
   LENGTH(customer_name) AS name_length
FROM customer;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	uppercase_name	lowercase_name	name_length
Rahul Patil	RAHUL PATIL	rahul patil	11
Priya Shah	PRIYA SHAH	priya shah	10
Amit Naik	AMIT NAIK	amit naik	9
Sneha Rao	SNEHA RAO	sneha rao	9

tup

Explanation: Demonstrates uppercase conversion, lowercase conversion, and name-length calculation

12. Use number functions

```
SELECT
   ticket_price,
   ROUND(ticket_price / 3, 2) AS divided_price,
   MOD(ticket_price, 2) AS price_remainder
FROM showtime;
```



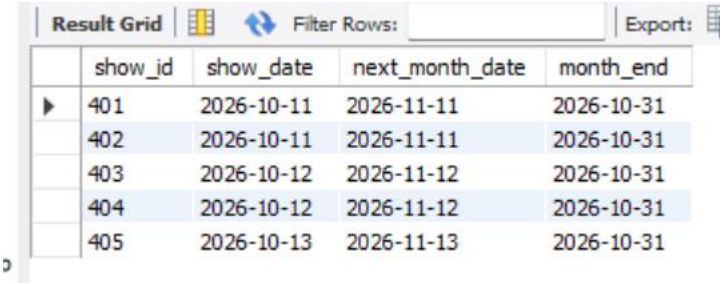
Result Grid | Filter Rows: | Expor

ticket_price	divided_price	price_remainder
250.00	83.33	0.00
200.00	66.67	0.00
300.00	100.00	0.00
180.00	60.00	0.00
220.00	73.33	0.00

Explanation: Demonstrates ROUND() and MOD() using ticket prices.

13. Use date functions

```
SELECT
   show_id,
   show_date,
   DATE_ADD(show_date, INTERVAL 1 MONTH) AS next_month_date,
   LAST_DAY(show_date) AS month_end
FROM showtime;
```



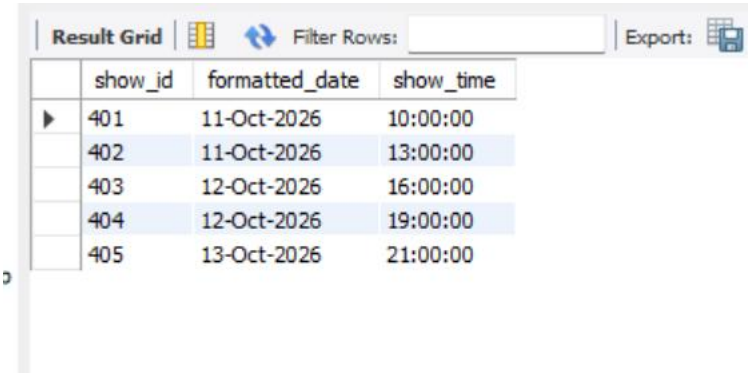
Result Grid | Filter Rows: | Export:

show_id	show_date	next_month_date	month_end
401	2026-10-11	2026-11-11	2026-10-31
402	2026-10-11	2026-11-11	2026-10-31
403	2026-10-12	2026-11-12	2026-10-31
404	2026-10-12	2026-11-12	2026-10-31
405	2026-10-13	2026-11-13	2026-10-31

Explanation: Displays the show date, the date one month later, and the last day of the show date's month**.**

14. Use conversion and general functions

```
SELECT
   show_id,
   DATE_FORMAT(show_date, '%d-%b-%Y') AS formatted_date,
   IFNULL(show_time, 'Not Available') AS show_time
FROM showtime;
```



Result Grid | Filter Rows: | Export:

show_id	formatted_date	show_time
401	11-Oct-2026	10:00:00
402	11-Oct-2026	13:00:00
403	12-Oct-2026	16:00:00
404	12-Oct-2026	19:00:00
405	13-Oct-2026	21:00:00

Explanation: Formats dates as text and replaces any NULL status with Not Available.

B.2. Observation

The SELECT statement was used to retrieve records from the Cinema Ticket Booking System tables. The WHERE clause restricted records according to specified conditions. The DISTINCT keyword eliminated duplicate values, and ORDER BY arranged records in ascending order. Comparison and logical operators were used to filter data. Character, number, date, and conversion functions were applied to retrieve and process information from the database.
Result: Basic SQL retrieval statements, selection, projection, sorting, and built-in functions were successfully practised using MySQL Workbench.

B2 - Write answers to the following-

Q1. Explain the **structure of a SELECT statement**. List and describe its main clauses.

The SELECT statement retrieves data from one or more database tables.

SELECT column_name

FROM table_name

WHERE condition

ORDER BY column_name;

- SELECT: Specifies the columns to retrieve.
- FROM: Specifies the table.
- WHERE: Filters rows according to a condition.
- ORDER BY: Sorts the result.
- DISTINCT: Removes duplicate rows from the result.

Q2. Differentiate between **DISTINCT and ALL** with suitable examples.

DISTINCT	**ALL**
Returns only unique values or rows.	Retains duplicate values or rows.
Eliminates duplicates.	Does not eliminate duplicates.
Must be specified when required.	Default behavior when neither keyword is specified.

Q3. Explain the role of the **WHERE clause** in SQL. How does it help in restricting data?

The WHERE clause filters records based on a specified condition. It returns only those rows that satisfy the condition.
Ex: -

```
SELECT *
FROM customer
WHERE city = 'Pune';
```

Q4. Differentiate between the operators used in WHERE clause:

- Comparison operators: =, >, <, >=, <=, <>. They compare values.
- Logical operators: AND, OR, NOT. They combine or reverse conditions.

Example:

```
SELECT *
FROM customer
WHERE city = 'Mumbai'
AND customer_id > 601;
```

Q5. What are **built-in (aggregate) functions** in SQL? Explain **COUNT(), SUM(), AVG(), MAX(), MIN()** with their purpose.

Aggregate functions calculate a result from a group of rows.

Function	**Purpose**	**Example**
COUNT()	Counts records or non-NULL values.	COUNT(*)
SUM()	Calculates the total.	SUM(ticket_price)
AVG()	Calculates the average.	AVG(ticket_price)
MAX()	Returns the highest value.	MAX(ticket_price)
MIN()	Returns the lowest value.	MIN(ticket_price)